FIPU Fakultet informatike u Puli · Sveučilište Jurja Dobrile u Puli

Raspodijeljeni sustavi: uputa za prijavu projekta

izv. prof. dr. sc. Nikola Tanković

Prije izrade projekt **prijavljujete**, a nastavnik ga **odobrava**. Prijava je kratak prijedlog sustava (1–2 stranice) iz kojega se vidi *što* gradite, *za koga* i *zašto* je to raspodijeljeni sustav. Poveznica za prijavu nalazi se na **E-Učenju** i na stranici kolegija **ntankovic.unipu.hr/rs**.

> **Raspodijeljeni sustav** čine komponente koje se neovisno izvršavaju te komuniciraju i koordiniraju razmjenom poruka radi zajedničkog cilja. Prijava treba pokazati koje su to komponente u vašem sustavu, koje poruke razmjenjuju i kako se koordiniraju.

Što prijava mora sadržavati

Naziv, tim i sažetak

Navedite naziv projekta, članove tima i sažetak od dvije do tri rečenice: koji problem sustav rješava i što korisnik na kraju dobiva. Ako radite u timu, ukratko navedite tko je odgovoran za koje komponente.

Korisnici i funkcionalnosti

- **Za koga je sustav namijenjen?** Opišite jednu ili dvije skupine korisnika (npr. studenti, nastavnici, analitičari, operateri sustava).
- **Koje funkcionalnosti pruža?** Navedite tri do šest glavnih funkcionalnosti kao rečenice iz perspektive korisnika: „Analitičar učitava skup podataka i pokreće obradu.”
- **Korisničko sučelje je obvezno** i ima dva dijela:
  - *sučelje za korištenje*, kroz koje krajnji korisnik obavlja glavne funkcionalnosti;
  - *sučelje za upravljanje*, kroz koje administrator ili operater prati stanje komponenti i poslova, pokreće ili zaustavlja obrade, pregledava pogreške i mijenja postavke.

  Oba dijela mogu biti u istoj web-aplikaciji s različitim ulogama. Navedite tehnologiju sučelja.

Zašto je sustav raspodijeljen

Ovo je najvažniji dio prijave. Navedite **barem jedan konkretan razlog** zbog kojeg sustav ne može (ili ne bi trebao) raditi kao jedan proces na jednom računalu:

- **količina podataka ili obrade:** posao se dijeli na više procesa koji rade paralelno;
- **opterećenje:** više istovremenih korisnika ili zahtjeva nego što jedan poslužitelj može obraditi;
- **dostupnost:** sustav mora nastaviti raditi kada jedna komponenta ispadne;
- **izvori podataka na više mjesta:** podaci nastaju ili se čuvaju na različitim lokacijama;
- **različiti zahtjevi komponenti:** npr. zahtjevna obrada ili inferencija modela odvojena je od API-ja kako ne bi usporavala korisnike.

Razlog potkrijepite pretpostavkom o opterećenju, npr. „200 istovremenih korisnika”, „10 GB zapisa dnevno” ili „obrada jednog skupa traje 20 minuta na jednom procesu”. Odgovorite i na pitanje: *što bi se dogodilo kada bi sve radilo na jednom računalu?*

> **Nisu dovoljni razlozi:** „jer su mikroservisi moderni”, „jer koristimo Docker” ili podjela jednostavne aplikacije na usluge koje uvijek pozivaju jedna drugu i nemaju vlastitu odgovornost.

Komponente sustava

Nabrojite komponente koje se izvršavaju kao zasebni procesi (kontejneri). Za svaku navedite odgovornost, podatke za koje je zadužena i tehnologiju. Dovoljna je kratka tablica:

Komponenta	Odgovornost	Stanje (podaci)	Tehnologija
API-pristupnik	prima zahtjeve sučelja, provjerava korisnika	nema vlastito stanje	FastAPI
Usluga poslova	stvara i prati poslove obrade	tablica poslova	FastAPI, PostgreSQL
Procesi obrade	paralelno obrađuju dijelove podataka	međurezultati	Spark

Priložite **dijagram arhitekture**: okviri su komponente, strelice su komunikacija. Na strelicama označite je li komunikacija sinkrona ili asinkrona.

Komunikacija i koordinacija

Za svaku vezu između komponenti objasnite:

- **kako komuniciraju:** sinkrono (zahtjev–odgovor, npr. REST preko HTTP-a uz `httpx`) ili asinkrono (red poruka, objavi–pretplati se), te u kojem formatu (npr. JSON);
- **kako se koordiniraju:** tko dodjeljuje posao (npr. Spark driver, red poruka iz kojega procesi preuzimaju zadatke), kako komponente pronalaze jedna drugu (npr. imena usluga u Docker mreži) i tko je vlasnik zajedničkog stanja;
- **što se događa kada nešto pođe po zlu:** što komponenta radi pri isteku vremena čekanja (timeout), smije li ponoviti zahtjev (ponovni pokušaj, retry) bez dvostrukog učinka (idempotentnost) i kako sustav reagira na djelomični ispad, npr. kada jedan proces obrade padne usred posla.

Tehnologije

Koristite tehnologije obrađene na predavanjima ili one koje odobri nastavnik. Preporučene su:

Namjena	Preporučene tehnologije
usluge i API	Python, FastAPI, Uvicorn, `httpx`, `asyncio`
raspodijeljena obrada podataka	Apache Spark (PySpark)
poruke i događaji	red poruka ili objavi–pretplati se (npr. RabbitMQ, Redis)
pokretanje i orkestracija	Docker Compose, Docker Swarm
pohrana	relacijska ili NoSQL baza po izboru, uz obrazloženje

AI/ML je dopušten. Model strojnog učenja u prijavi tretirajte kao *komponentu sustava*: gdje se izvršava, kako mu stižu zahtjevi ili podaci, kako se skalira i što se događa kada je nedostupan. Algoritmi strojnog učenja i njihova točnost **nisu predmet ocjene** pa ih ne morate detaljno opisivati; dovoljno je navesti vrstu modela i biblioteku. Svaku tehnologiju izvan popisa navedite u prijavi radi odobrenja.

Kako će sustav skalirati

- Koje komponente skalirate **horizontalno** (više replika), a koje **vertikalno** (jače računalo)? Komponente bez vlastitog stanja najlakše je replicirati.
- Gdje očekujete **usko grlo** (npr. baza, jedan red poruka, jedan proces koji sve koordinira)?
- Kako se posao ili podaci **dijele** između replika ili procesa (npr. po korisniku, po datoteci, po vremenskom razdoblju)?
- Što se događa kada se broj replika poveća ili kada jedna replika ispadne?
- Kako ćete skaliranje **pokazati**, npr. pokretanjem više replika i mjerenjem vremena obrade ili broja obrađenih zahtjeva?

Ideje za projekt

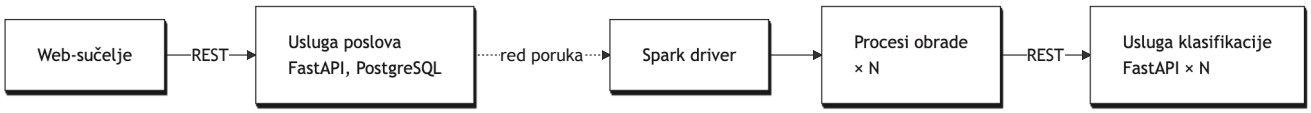
Ideje su poticaj za razmišljanje, a ne zadane teme. Prilagodite ih ili predložite vlastitu.

Ideja	Zašto je raspodijeljeno	Što ćete morati objasniti

Ideja	Zašto je raspodijeljeno	Što ćete morati objasniti
Prikupljanje očitanja senzora uz automatsko skaliranje u Docker Swarmu i NoSQL bazu (npr. MongoDB, Cassandra)	mnogo istovremenih upisa; opterećenje se mijenja tijekom dana	Swarm nema ugrađeno automatsko skaliranje: vlastita usluga prati opterećenje i mijenja broj replika. Kako se podaci dijele i repliciraju u bazi?
Javni red vožnje ili katalog kolegija s PostgreSQL-om u postavu primarna baza i replike za čitanje (primary–replica)	malo upisa, puno čitanja; čitanja se raspoređuju na replike	Koliko kasne replike i kada korisnik vidi zastarjeli podatak? Što se događa ako primarna baza ispadne?
Prijava na radionice ili mala web-trgovina kao mikroservisi, svaki sa svojom bazom (korisnici, narudžbe, plaćanje, obavijesti)	usluge se razvijaju, pokreću i skaliraju neovisno	Kako usluge razmjenjuju događaje (red poruka)? Kako očuvati pravilo koje obuhvaća dvije usluge, npr. da se mjesto ne proda dvaput?
Analiza velikog skupa podataka u Sparku nad Parquet datotekama u S3-kompatibilnoj pohrani (npr. MinIO), poput javnih podataka o vožnjama taksija	podaci su preveliki za jedan proces; obrada se dijeli po particijama	Kako se podaci dijele na procese i kako se objedinjuju rezultati? Što se događa kada jedan proces padne usred obrade?
Obrada slika ili dokumenata modelom strojnog učenja s redom poruka i više procesa za inferenciju	inferencija je spora i zahtjevna; procesi se dodaju prema duljini reda	Što ako proces preuzme zadatak i padne? Kako spriječiti dvostruku obradu istog zadatka?
Raspodijeljeni web-pretraživač (crawler) s koordinatorom i procesima koji preuzimaju adrese	puno sporih mrežnih zahtjeva; posao se dijeli po domenama	Kako izbjeći da dva procesa obrade istu adresu? Kako ograničiti broj zahtjeva prema jednoj domeni?

Primjer kratke prijave

Analiza recenzija proizvoda. Trgovci učitavaju velike CSV-datoteke recenzija i dobivaju izvještaj o raspoloženju kupaca; operater kroz sučelje za upravljanje prati poslove, procese obrade i pogreške. *Zašto raspodijeljeno:* datoteka ima do 5 milijuna recenzija pa Spark dijeli obradu na više procesa, a klasifikacija je zasebna usluga koja se replicira neovisno o API-ju. *Koordinacija:* usluga poslova šalje zadatak u red poruka, a Spark driver dodjeljuje dijelove podataka procesima. Svaki posao ima jedinstveni identifikator pa ponovno slanje ne stvara duplikat.



Puna strelica: sinkroni poziv ili dodjela posla. *Isprekidana strelica:* asinkrona poruka. Primjer je ilustracija, a ne obvezna arhitektura.

Obrana i usmeni ispit

Na obrani objašnjavate *svoj* sustav pojmovima s predavanja: zašto je raspodijeljen, kako komponente komuniciraju i što se događa pri ispadu. Usmeni ispit drži se **gradiva predavanja**, a ne detalja algoritama strojnog učenja ili pojedinih biblioteka.

Kontrolni popis prije predaje prijave

- ☐ naziv, članovi tima i sažetak od dvije do tri rečenice
- ☐ korisnici i tri do šest glavnih funkcionalnosti
- ☐ sučelje za korištenje i sučelje za upravljanje
- ☐ barem jedan konkretan razlog za raspodjelu i pretpostavka o opterećenju
- ☐ tablica komponenti i dijagram arhitekture
- ☐ način komunikacije i koordinacije te ponašanje pri isteku vremena i ispadu
- ☐ popis tehnologija, uz napomenu o onima koje nisu na popisu preporučenih
- ☐ plan skaliranja: što se replicira, gdje je usko grlo i kako ćete skaliranje pokazati